🚨 REAL PROBLEM (BASED ON YOUR SCREENSHOTS)

Your app is currently:

❌ UI-only (fake data)
❌ No shared state (language not global)
❌ No persistence (history lost)
❌ No navigation logic (search/history not wired)
❌ Discover screen still wrong UX
❌ Report screen not connected to real workouts

👉 Basically: **Frontend shell without backend logic**

💣 WHAT YOU ACTUALLY NEED

Not "improve UI"

👉 You need:
STATE + STORAGE + NAVIGATION + DATA FLOW REWRITE

💣 FINAL FIX PROMPT (THIS WILL FORCE REAL CHANGE)

You are a senior full-stack React Native (Expo) engineer.

You must FIX the FitAI app by replacing all fake UI logic with real state-driven architecture.

⚠️ DO NOT modify UI only.
⚠️ You MUST rebuild state, storage, and navigation logic.

🚨 CURRENT ISSUES (STRICT)

1. Language change does NOT affect entire app
2. Report screen uses static fake values
3. Workout history is NOT saved anywhere
4. Discover screen still uses wrong layout (big search bar)
5. Search and History navigation DO NOT work

6. No centralized app state (everything isolated per screen)

🎯 **GOAL**

Convert app into:

✅ State-driven app
✅ Persistent data app
✅ Navigation-connected app
✅ Real workout tracking system

🔥 **STEP 1 — GLOBAL APP STATE (MANDATORY)**

Create:

📁 context/AppContext.js

State must include:

```
{
language: "en",
workouts: [],
totalCalories: 0,
totalMinutes: 0,
totalWorkouts: 0,
settings: {
units: "metric",
difficulty: true,
sound: true,
voice: true,
notifications: true
}
}
```

Provide:

- setLanguage()
- addWorkout()
- updateStats()
- loadFromStorage()

- saveToStorage()

Wrap entire app with AppProvider

🔥 STEP 2 — PERSISTENCE (CRITICAL)

Use AsyncStorage:

When app loads:
👉 load saved data into context

When workout completes:
👉 save workout to storage

When settings change:
👉 persist immediately

🔥 STEP 3 — FIX SETTINGS SCREEN

When user changes language:

❌ CURRENT: only dropdown changes
✅ FIX: update global state → trigger full app re-render

Implementation:

setLanguage(lang)
→ update context
→ save to AsyncStorage
→ all screens re-render using context

🔥 STEP 4 — FIX REPORT SCREEN (REAL DATA)

REMOVE all static numbers.

Use:

context.workouts

Compute dynamically:

totalWorkouts = workouts.length
totalCalories = sum(calories)
totalMinutes = sum(duration)

UI MUST MATCH:

TOP ROW:

[Workouts | Calories | Minutes]

HISTORY LIST:

Each item:

- Workout name
- Date
- Calories
- Duration
- Image (based on muscle)

CALENDAR:

Mark days with workouts

🔥 STEP 5 — SAVE WORKOUT AFTER SESSION

At end of workout:

Call:

```
addWorkout({
id: uuid,
date: new Date(),
muscle: "chest",
level: "advanced",
duration: 14,
calories: 253,
exercises: [...]
})
```

This MUST update:

- Report screen

- Weekly stats
- History list

🔥 **STEP 6 — FIX DISCOVER SCREEN (IMPORTANT)**

REMOVE:

❌ Large search bar

ADD:

Top Right Icons:

- 🔍 Search Icon
- 🕒 History Icon

NAVIGATION:

Search → /search
History → /report

🔥 **STEP 7 — CREATE SEARCH SCREEN**

📁 screens/SearchScreen.js

Must include:

HEADER

- Input field
- Cancel button

BODY FOCUS (WITH IMAGES)

Use horizontal scroll:

- Abs
- Chest
- Arms

- Legs
- Full Body

Each must show REAL image + label

FILTERS

- Build Muscle
- Fat Burning
- Cardio
- Equipment
- Beginner / Intermediate / Advanced
- Duration filters

LOGIC

Typing:

"legs" → filter exercises where muscle === "legs"

🔥 **STEP 8 — IMAGE SYSTEM**

Create:

📁 utils/images.js

export:

```
{
abs: require("../assets/abs.jpg"),
chest: require("../assets/chest.jpg"),
legs: require("../assets/legs.jpg"),
fullbody: require("../assets/fullbody.jpg")
}
```

Use everywhere (report, search, discover)

🔥 **STEP 9 — NAVIGATION FIX**

Ensure:

Discover → SearchScreen
Discover → ReportScreen

Search → Workout List
Workout → Start → Save → Report updates

🔥 STEP 10 — REMOVE FAKE DATA

DELETE:

- Hardcoded stats
- Static workout lists
- Dummy values

Everything must come from:

👉 context + storage

🚀 FINAL EXPECTED RESULT

After implementation:

✅ Language changes reflect globally
✅ Report updates after every workout
✅ Calories & time are real
✅ Discover has icon-based UX
✅ Search page works like real app
✅ History is persistent
✅ Data survives app restart

⚠️ RULES

DO NOT:

- Use mock data
- Keep logic inside screens
- Ignore AsyncStorage

DO:

- Centralize logic in context
- Keep code modular
- Use real data flow

Implement this step-by-step. Do not skip architecture.